LiquidHaskell: Experience with Refinement Types in the Real World

Niki Vazou, Eric L. Seidel, Ranjit Jhala

Szabó Márk

Haskell Background

Haskell's Type System

Haskell is a **statically typed** language: every expression has a type, and types are checked before the program runs.

Some common built-in types:

```
Int       -- integer
Float     -- floating point number
Double    -- floating point number
Num       -- [Int, Float, Double, ...]
Bool      -- True or False
Char      -- single character
String    -- text
```

Function Types

The arrow (→) separates input and output types.

```
add :: Int -> Int -> Int
add x y = x + y
```

Every function takes **one argument**: `add` takes `x` and returns a function that takes `y`.

A C-like version gives the **same result**, but is not logically the same, since it takes both arguments at once:

```
int add(int x, int y) { return x + y; }
```

Limits of The Type System

We can’t define constraints for the types of a function like:

- Conditions about the value: `x > 0`
- Relationships between values: `a > b`

To check that the input is non-negative, we need a dynamic test:

```haskell
sqrtRoot :: Double -> Double
sqrtRoot x
  | x >= 0    = sqrt x
  | otherwise = error "x must be non-negative"
```

Refinement Types and LiquidHaskell

LiquidHaskell

Many of these properties and relationships can be verified using a combination of **Refinement Types** and **external SMT solvers**.

This paper introduces and experiments with **LiquidHaskell** a refinement typechecker used to verify more than **10,000** lines of Haskell code across popular libraries.

Including: *bytestring*, *text*, *vector-algorithms* and *xmonad*.

Optional checker

Refinement types are written inside `{-@ ... @-}` comments, so normal Haskell compilers ignore them. Verification happens before the program runs and adds no runtime checks.

Refinement Types

Refinement types:

- contain a **predicate** which is assumed to be true for any element of the refined type.
- can express **preconditions** when used as function arguments or **postconditions** when used as return types.

Examples:

positive natural numbers

$x : \{n \in \mathbb{N} \mid n > 0\}$

result is a positive number

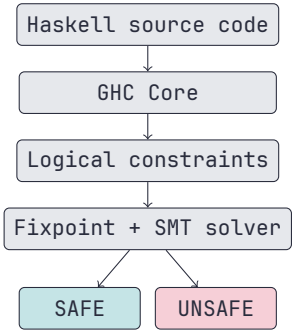
$f : \mathbb{N} \to \{y \in \mathbb{N} \mid y > 0\}$

result > input

$f : (x : \mathbb{N}) \to \{y \in \mathbb{N} \mid y > x\}$

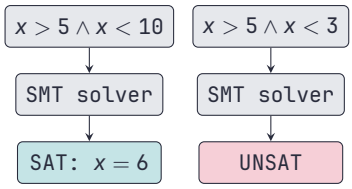
LiquidHaskell Architecture

- **Core:** GHC translates Haskell into a simpler intermediate representation; source locations are kept for error reporting.
- **Constraints:** refinement types become logical constraints that must hold.
- **Solution:** a fixpoint algorithm and an SMT solver (e.g. *Z3*, *CVC4*) check whether the constraints can be satisfied.
- **Result:** **SAFE**, or **UNSAFE** with errors mapped back to source locations.



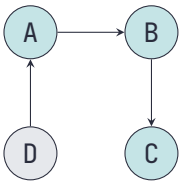
SMT Solvers

- An SMT solver takes a logical formula and answers whether values exist that make it true.
- **SAT**: yes, and it gives an example.
- **UNSAT**: no such values exist.
- LiquidHaskell asks whether a constraint can be **violated**: UNSAT means it always holds, so the program is **SAFE**.



Fixpoints

- A **fixpoint** of f is a value that f does not change: $f(x) = x$.
- To find one, apply f again and again until the result stays the same.



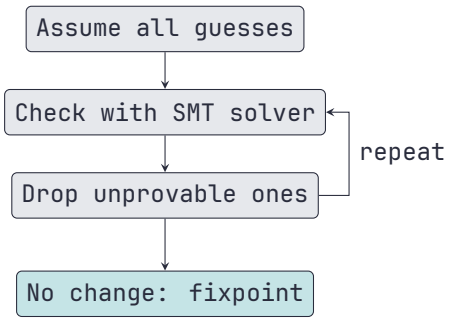
Example: which nodes can we reach from A?
Each step adds the next nodes along the arrows.



no change: fixpoint

Fixpoint in LiquidHaskell: Inferring Refinements

- The user writes only some refinements. For helper functions and local values LiquidHaskell has to **infer** them.
- Guesses come from the user's refinements and unprovable ones are dropped.
- Recursion links the checks, so this repeats until nothing changes.



LiquidHaskell Basics

```
1  {-@ type Natural = {v:Int | v >= 0} @-}
2  {-@ type Pos = {v:Int | v > 0} @-}
3
4  {-@ divide :: n:Natural -> d:Pos -> {v:Natural | v <= n} @-}
5  divide :: Int -> Int -> Int
6  divide x y = x `div` y
7
8  good = divide 5 2
9
10 -- With LH -> Type Mismatch before executing the code
11 -- Without LH -> runtime error: divide by zero
12 bad = divide 5 0
```

LiquidHaskell Parameterized Types

Value parameters are written in uppercase: here `Low` and `High`.

```
1  {-@ type Rng Low High = {v:Int | v >= Low && v <= High} @-}
2  {-@ grade :: Rng 0 100 -> Int @-}
3  grade :: Int -> Int
4  grade score
5      | score >= 90 = 5
6      | score >= 75 = 4
7      | score >= 60 = 3
8      | score >= 50 = 2
9      | otherwise   = 1
10 good = grade 85
11 bad = grade 120
```

Refining Data and Interfaces

LiquidHaskell Measures

So far, the specifications have been limited to comparisons and arithmetic operations on **primitive** values. **Measures** describe **inductive properties** of algebraic data types.

```
1 {-@ measure myLen @-}
2 myLen :: [a] -> Int
3 myLen []     = 0
4 myLen (x:xs) = 1 + myLen xs
```

- One equation for each data constructor: `[]` and `(:)`.
- Each right-hand side must belong to the restricted refinement logic.
- `measure myLen` makes this Haskell function available in refinements.

LiquidHaskell Measures: Minimum Length

```
1 {-@ type MinLength a N = {v:[a] | myLen v >= N} @-}
2
3 {-@ good :: MinLength Int 3 @-}
4 good :: [Int]
5 good = [1, 2, 3, 4]
6
7 {-@ bad :: MinLength Int 3 @-}
8 bad :: [Int]
9 bad = [1, 2]
```

- `a` is the element type; `N` is the minimum length.
- **SAFE:** `good` has length 4, and $4 \geq 3$.
- **UNSAFE:** `bad` has length 2, and $2 < 3$.

LiquidHaskell Refined Data Types

Every value of a data type may need to satisfy an **invariant**.

```
1 data CSV a = CSV { cols :: [String], rows :: [[a]] }
```

- `cols`: the column names in the header.
- `rows`: the table's rows, containing values of type `a`.
- Haskell allows rows of different lengths.

CSV invariant

Every row must have exactly as many entries as the header has column names.

LiquidHaskell Refined Data Types: Definition

A field's refinement can refer to an **earlier field**: each row must have the same length as `cols`.

```
{-@ type ListL a X = {v:[a] | len v == len X} @-}

{-@ data CSV a = CSV { cols :: [String]
                     , rows :: [ListL a cols] } @-}
```

```
good = CSV ["Month", "Days"]
  [ ["Jan", "31"]
  , ["Feb", "28"]
  , ["Mar", "31"] ]
```

```
bad = CSV ["Month", "Days"]
  [ ["Jan", "31"]
  , ["Feb", "28"]
  , ["Mar"] ]
```

LiquidHaskell Refined Type Classes

A **type class** defines a shared interface. Refinements give the method a **shared contract**:

```
{-@ class Reward a where
      reward :: a -> points:{Int | points >= 0}
             -> {v:Int | v >= points}
@-}
```

- **Precondition:** input points must be non-negative.
- **Postcondition:** the result cannot be smaller than the input.
- Every instance must satisfy the **same contract**.

LiquidHaskell Refined Type Classes: Verification

Instances must prove the postcondition $v \geq points$:

SAFE: adds a bonus

```
1 instance Reward Bool where
2   reward bonus points
3     | bonus     = points + 10
4     | otherwise = points
```

UNSAFE: deducts points

```
1 instance Reward Bool where
2   reward bonus points
3     | bonus     = points - 10
4     | otherwise = points
```

Callers must meet the precondition $points \geq 0$:

```
1 good = reward True 20       -- returns 30
2 bad = reward True (-1)
```

Termination

LiquidHaskell Termination

Haskell only evaluates expressions when their results are needed (*non-strict evaluation*).

- Some functions may run forever without producing a result.
- **LiquidHaskell:** Must ensure that functions terminate to safely verify certain properties.
- By default, LiquidHaskell proves termination of each recursive function.

Termination metric

A termination metric is a value that:

- Gets smaller with **every** recursive call.
- Cannot decrease forever (e.g., a non-negative integer).

LiquidHaskell Termination: Factorial

A simple termination metric is a **non-negative integer**.

```
1 {-@ fac :: n:Nat -> Nat / [n] @-}
2 fac :: Int -> Int
3 fac 0 = 1
4 fac n = n * fac (n - 1)
```

- `Nat`: integers greater than or equal to zero.
- `/ [n]`: use the argument `n` as the termination metric.
- The base case returns immediately.
- The recursive case decreases `n` by 1 each call.

LiquidHaskell Termination: Processing Batches

Consider a simplified **batch worker** that counts completed jobs.

- `b`: batches remaining, including the current batch.
- `j`: jobs remaining in the current batch.
- Completing a job decreases `j`, while `b` stays unchanged.
- Moving to the next batch decreases `b` and resets `j` to 3.

Two batches of three jobs

$(2,3) \to (2,2) \to (2,1) \to (2,0) \to (1,3) \to \cdots \to (1,0)$.

Neither argument strictly decreases at every call. We supply a **custom termination metric**: the pair `[b, j]`.

LiquidHaskell Termination: Lexicographic Ordering

The metric is written in the function's signature:

```
{-@ jobs :: b:Nat -> j:Nat -> Nat / [b, j] @-}
```

`/ [b, j]` uses **lexicographic ordering**: compare `b` first, and compare `j` only when `b` is unchanged.

- Completing a job, $(b, j) \to (b, j-1)$: `b` stays equal, `j` decreases.
- Next batch, $(b, 0) \to (b-1, 3)$: `b` decreases, so `j` may grow.
- Both components stay non-negative, so the pair cannot decrease forever.

Functional Correctness Invariants

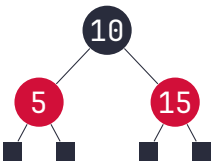
LiquidHaskell Functional Correctness: Red-Black Trees

Beyond avoiding errors, refinements can express **application-specific correctness rules**. The paper's example: **red-black trees**, a balanced binary search tree.

- **Order:** all keys in the left subtree are smaller than the node's key and all keys in the right subtree are larger.
- **Color:** a red node has only black children. Empty leaves count as black.
- **Height:** from any node, every path to a leaf contains the same number of black nodes.

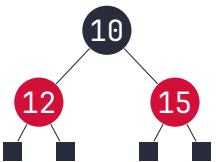
LiquidHaskell Red-Black Trees: Valid and Invalid

Valid



$5 < 10 < 15$.
Color and height are valid too.

Invalid



Order fails: $12 > 10$.

Black squares represent empty leaves.

LiquidHaskell Red-Black Trees: Checking the Invariants

- **Color** and **height** are checked by **measures**: recursive functions over the tree, like `myLen` earlier.
- **Order** is built into the tree type itself: its parameters require smaller keys on the left and larger keys on the right.
- The valid red-black tree type combines all three, without changing the underlying tree structure.

Result

LiquidHaskell verifies the library's **insertion, deletion, and rebalancing** procedures.

Evaluation

LiquidHaskell Evaluation

- **11,512** lines of code in 56 modules, checked with **1,975** lines of specifications.
- Hints (annotations and qualifiers) add only **3.5%** of the code size.

Performance

The full suite takes about **22 minutes**, but checking a handful of functions in isolation is fast enough for **interactive use**.

LiquidHaskell Limitations

The case studies also showed limitations of LiquidHaskell.

- **Ghost parameters:** values needed only for the proof must still be passed as extra arguments.
- **Fixed-width numbers:** integers are treated as unbounded and floats as exact reals, so overflow and rounding are not captured.

The paper lists several **further limitations** as well.

Workaround

In most cases, the code could be **altered slightly** to make verification possible.

References

- Niki Vazou, Eric L. Seidel, Ranjit Jhala. *LiquidHaskell: Experience with Refinement Types in the Real World*. Haskell Symposium, 2014.
- LiquidHaskell documentation: https://ucsd-progsys.github.io/liquidhaskell
- Satisfiability modulo theories: https://en.wikipedia.org/wiki/Satisfiability_modulo_theories